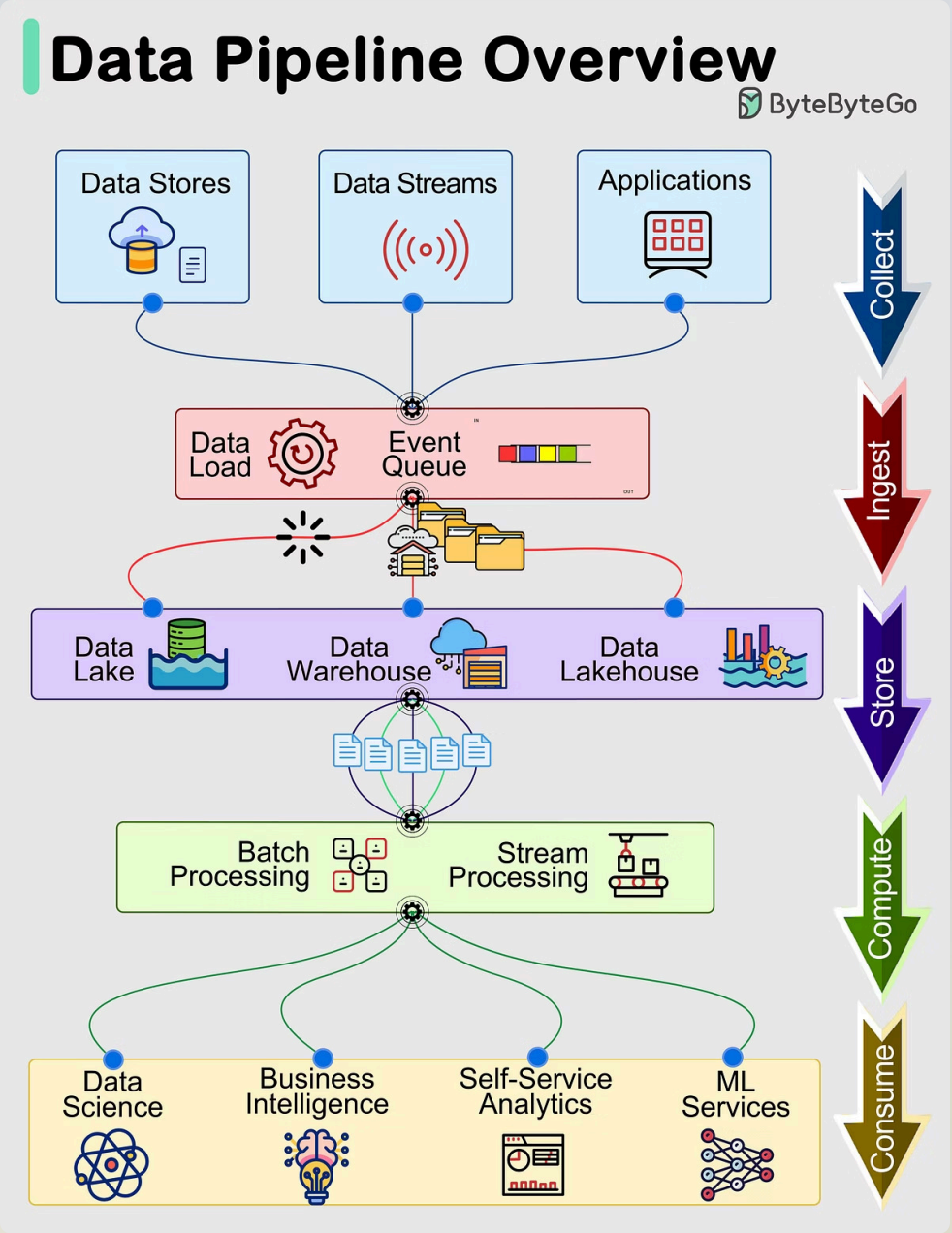
Batch Processing con Apache Spark



Objetivos de Aprendizaje

01

Diferenciar batch vs streaming

Ubicar Apache Spark en el ecosistema de datos moderno y comprender cuándo aplicar cada paradigma de procesamiento

02

Modelo de ejecución distribuida

Dominar la arquitectura de jobs → stages → tasks y el concepto de evaluación perezosa para optimizar el rendimiento

03

APIs de Spark

Conocer RDD, DataFrame y Dataset, identificando cuándo usar cada abstracción según el caso de uso específico

04

Formatos de almacenamiento

Entender el impacto de formatos como Parquet en el rendimiento y las estrategias de optimización de datos

05

Patrones y buenas prácticas

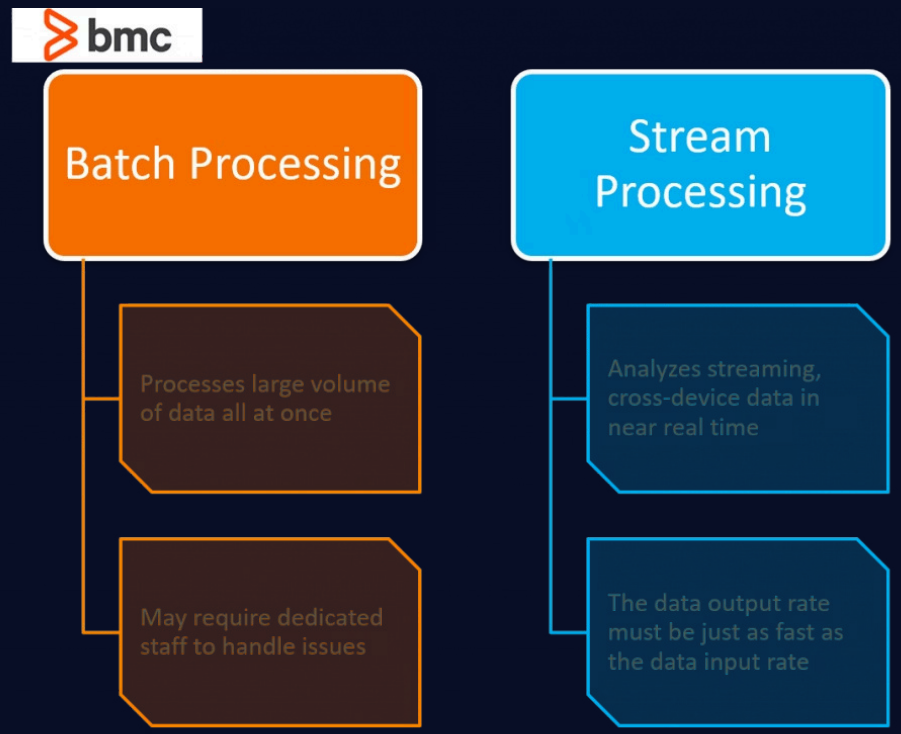
Interiorizar técnicas de particionado, joins, shuffle y caching para construir pipelines de datos eficientes

¿Qué es Procesamiento Batch?

Definición y características

El procesamiento batch maneja datos "fríos" en lotes programados, con latencias de minutos u horas. Es ideal para operaciones que no requieren respuesta inmediata pero sí alto throughput.

- Procesa grandes volúmenes de datos históricos
- Ejecuta en ventanas temporales definidas
- Optimizado para eficiencia de recursos
- Tolerante a latencias altas



Benefits and drawbacks of common data processing types

Casos de uso típicos

- **ETL diarios:** transformación de datos transaccionales
- **Feature stores offline:** preparación de características para ML
- **Cierres contables:** consolidación financiera
- **Agregaciones históricas:** reportes y analíticas

Apache Spark en el Ecosistema

Motor distribuido unificado

Spark funciona como motor tanto para batch como streaming estructurado, proporcionando una API consistente across paradigmas

Paralelismo masivo

Distribuye automáticamente el trabajo en clusters de cientos o miles de núcleos, maximizando el uso de recursos

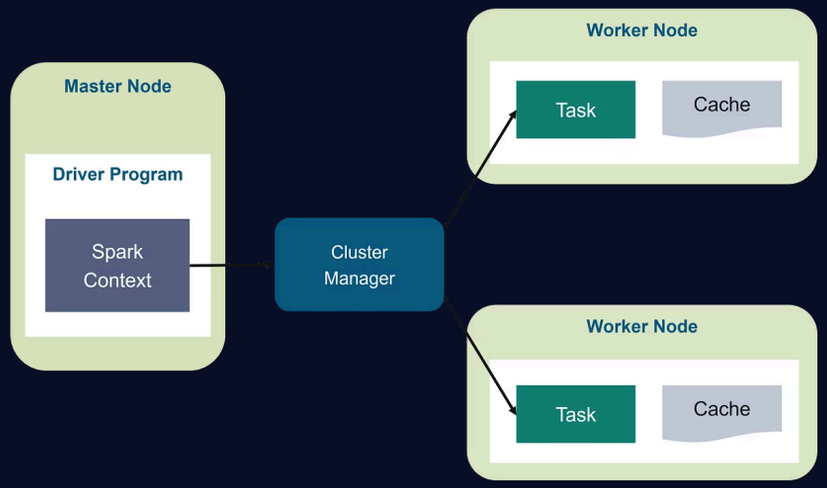
Tolerancia a fallos

Recupera automáticamente de errores mediante el linaje de transformaciones, sin requerir checkpoints manuales

API unificada

Soporta SQL, DataFrame y RDD desde el mismo motor, facilitando la adopción y el mantenimiento

Arquitectura de Alto Nivel



1

Driver Program

Coordina la ejecución, mantiene el SparkContext y planifica las tareas. Contiene la lógica principal de la aplicación

2

Cluster Manager

Gestiona recursos del cluster (YARN, Kubernetes, Standalone). Asigna executors según la demanda

3

Executors

Procesos que ejecutan tareas y almacenan datos en caché. Reportan estado y resultados al driver

La **Web UI** proporciona métricas detalladas de jobs, stages y tasks, siendo fundamental para diagnósticos de rendimiento y debugging de aplicaciones distribuidas.

Modelo de Ejecución: Jobs → Stages → Tasks

Job

Conjunto de transformaciones disparado por una acción (collect, save). Representa una unidad completa de trabajo

Stage

Subdivisión del job basada en dependencias de shuffle. Agrupa transformaciones que pueden ejecutarse en paralelo

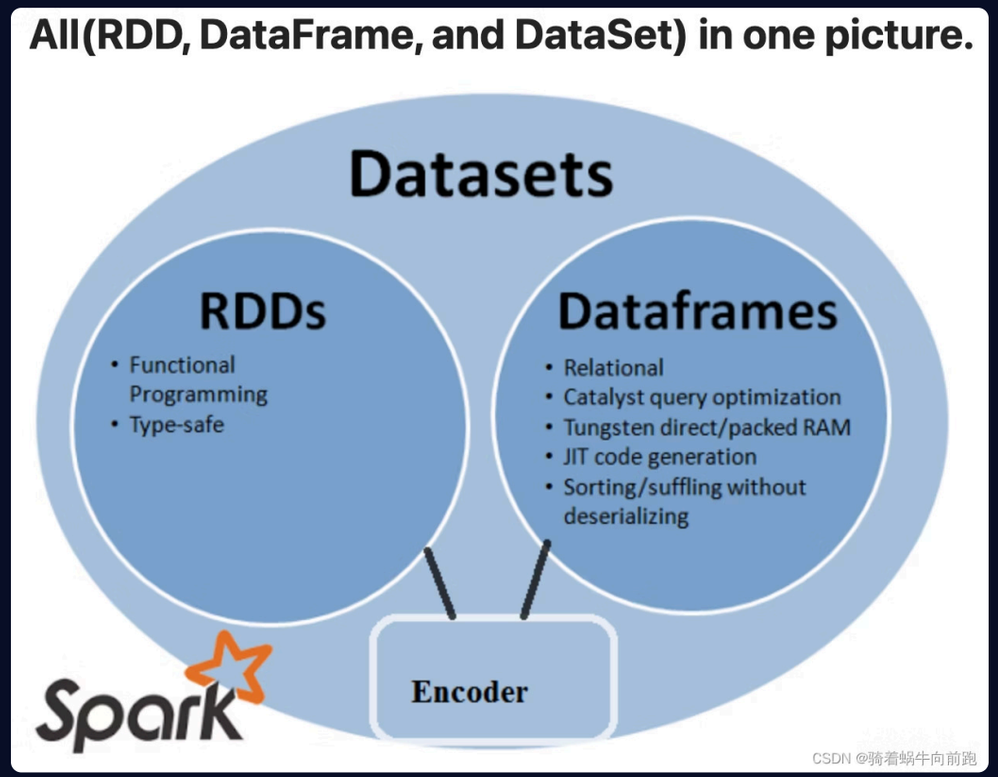
Task

Unidad mínima de trabajo ejecutada en una partición. Múltiples tasks por stage procesan diferentes particiones

La **evaluación perezosa** significa que las transformaciones solo construyen un plan de ejecución (DAG), pero la ejecución real ocurre cuando se invoca una acción.

APIs de Spark: RDD vs DataFrame vs Dataset

RDD (Resilient Distributed Dataset)

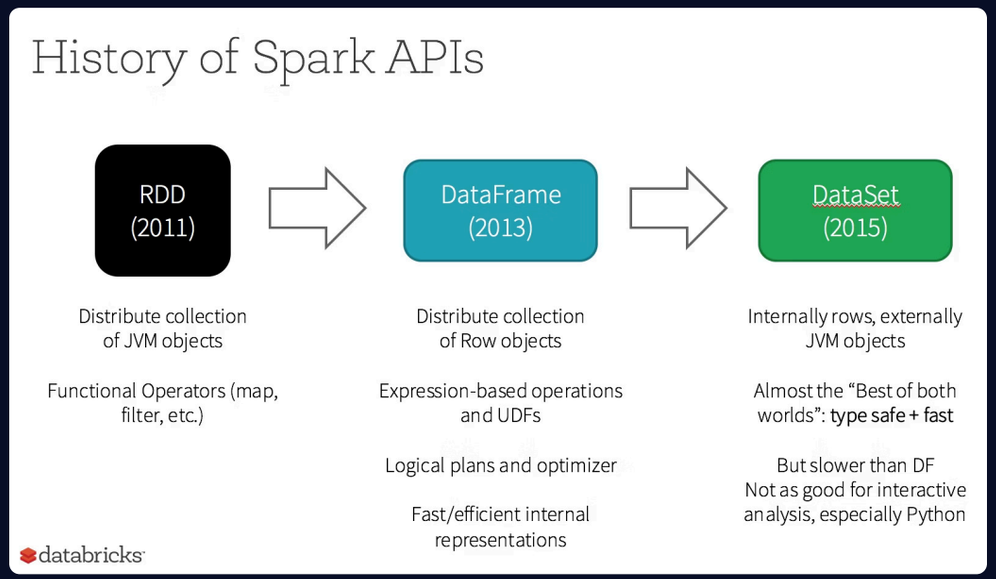


Abstracción de bajo nivel que ofrece control fino sobre particiones y funciones personalizadas. Incluye linaje automático para tolerancia a fallos.

Cuándo usar:

- Operaciones muy personalizadas
- Control específico de particionado
- Funciones complejas no soportadas en DF/DS

DataFrame/Dataset



APIs de alto nivel con esquema tipado y optimización automática vía Catalyst Optimizer. Recomendadas para analítica tabular.

Cuándo usar (recomendado):

- Operaciones SQL/analíticas estándar
- Proyecciones, filtros, joins, agregaciones
- Máximo rendimiento con optimización automática

Ejemplo Práctico: RDD vs DataFrame

Código con RDD (bajo nivel)

```
# RDD - Control manual
rdd = spark.sparkContext.textFile("logs.txt")

filtered_rdd = rdd.filter(lambda line: "ERROR" in line)

word_counts = filtered_rdd.flatMap(lambda line: line.split()) \
.map(lambda word: (word, 1)) \
.reduceByKey(lambda a, b: a + b)

result = word_counts.collect()
```

Código con DataFrame (optimizado)

```
# DataFrame - Optimización automática
df = spark.read.text("logs.txt")

filtered_df = df.filter(df.value.contains("ERROR"))
word_counts = filtered_df.select(
    explode(split(df.value, " ")).alias("word")
).groupBy("word").count()

result = word_counts.collect()
```

DataFrame aprovecha el **Catalyst Optimizer** para generar código optimizado y realizar pushdown de filtros, mientras RDD ejecuta exactamente el código especificado.

Ejecución Distribuida en Detalle

DAG Scheduler

Convierte el plan lógico (linaje de transformaciones) en plan físico dividido por stages, basándose en dependencias narrow vs wide

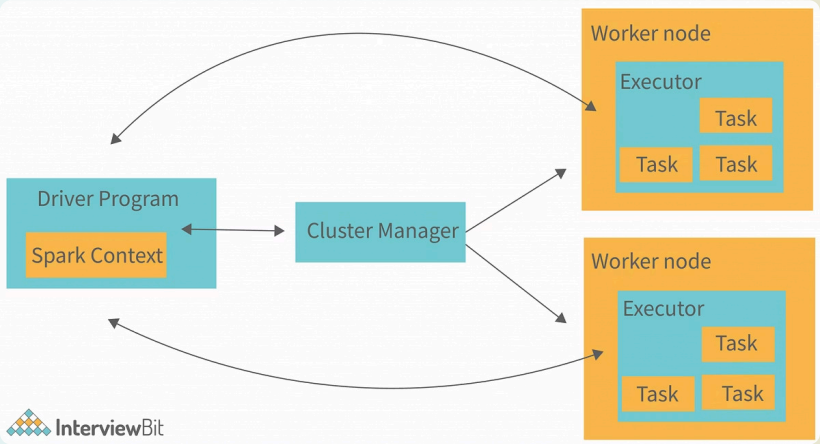
Task Scheduler

Lanza tasks individuales por partición, maneja reintentos de fallos y optimiza localidad de datos (local vs remoto)

Shuffle Operations

Barrera entre stages que requiere intercambio de datos por red. Genera métricas clave: Shuffle Read/Write

Clave de optimización: Minimizar operaciones de shuffle suele proporcionar las mejores mejoras de rendimiento en aplicaciones Spark.



Formatos de Datos: Por qué Parquet

Ventajas del formato Parquet

Almacenamiento columnar

Organiza datos por columnas, permitiendo leer solo las columnas necesarias (column pruning)

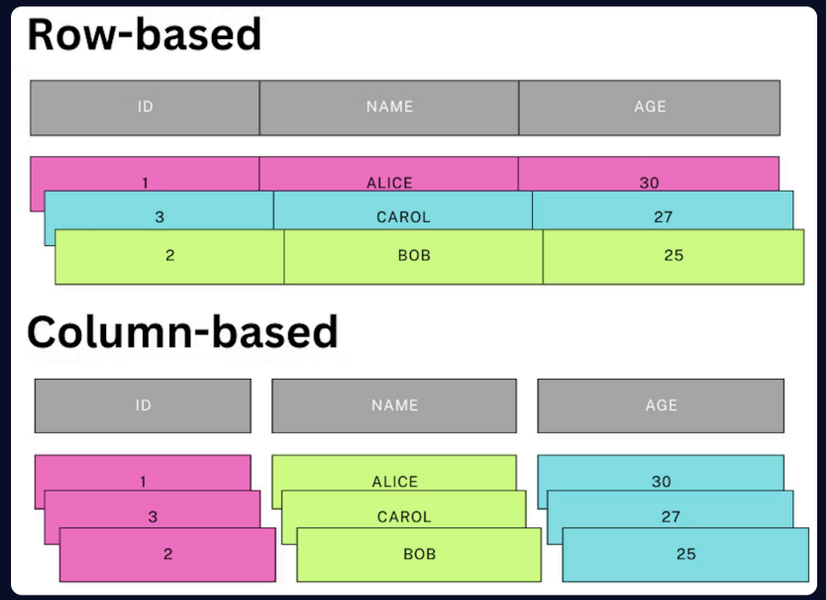
Compresión eficiente

Algoritmos como RLE y bit-packing reducen significativamente el tamaño de archivos

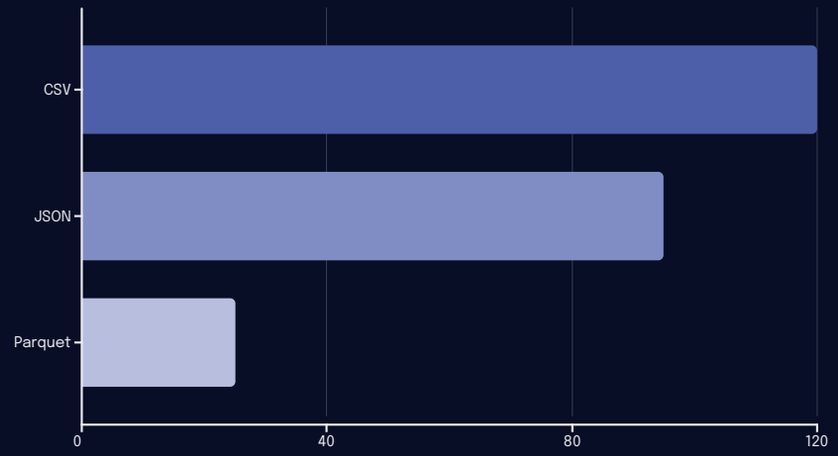
Predicados pushdown

Filtra datos a nivel de archivo usando metadatos, evitando leer bloques innecesarios

En Spark, Parquet permite **vectorización** y optimizaciones automáticas que CSV no puede ofrecer.



Comparación de rendimiento



Transformaciones Clave en Batch

Selección y Filtrado

Column pruning elimina columnas innecesarias. Predicate pushdown aplica filtros antes de leer datos, reduciendo IO significativamente.

Joins Distribuidos

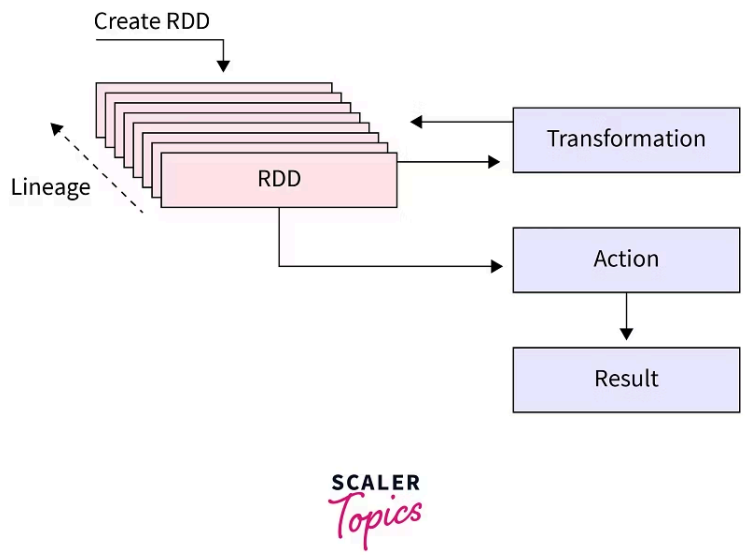
Broadcast joins para tablas pequeñas vs grandes. Sort-merge joins para tablas grandes. Shuffle-hash para casos específicos.

Agregaciones

groupBy optimizado con pre-agregación local. Window functions para análisis por particiones lógicas de datos.

Funciones Definidas por Usuario

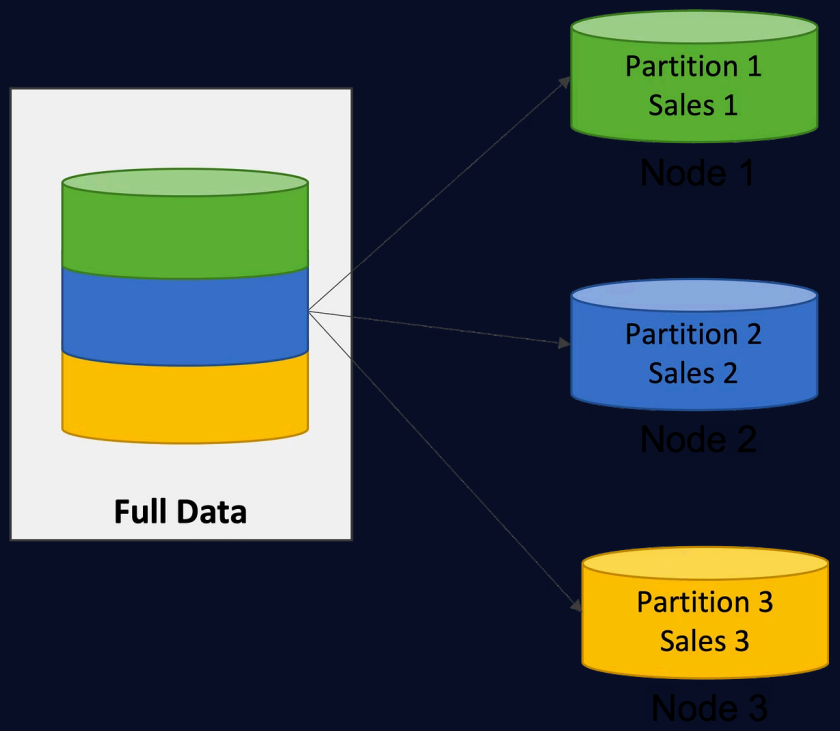
UDFs proporcionan flexibilidad pero sacrifican optimización. Preferir expresiones SQL nativas cuando sea posible.



Ejemplo: Optimizando Joins

```
# Forzar broadcast join para tabla pequeña
from pyspark.sql.functions import broadcast

# Tabla grande: transacciones (millones de filas)
transactions = spark.read.parquet("transactions.parquet")

# Tabla pequeña: catálogo de productos (miles de filas)
products = spark.read.parquet("products.parquet")

# Join optimizado con broadcast
result = transactions.join(
    broadcast(products),
    transactions.product_id == products.id,
    "inner"
)

# Alternativa: configuración automática
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

El **broadcast join** evita shuffle cuando una tabla es pequeña (<200MB por defecto), distribuyendo la tabla menor a todos los executors.



Particionado y Data Layout

Principios de particionado

Un particionado adecuado reduce data skew y mejora paralelismo. Evitar micro-particiones excesivas que generen overhead administrativo.

repartition() vs coalesce()

`repartition(n)` redistribuye datos con shuffle completo. `coalesce(n)` reduce particiones sin shuffle, más eficiente para consolidar.

Bucketing y ordenamiento

Escribir datos pre-organizados por bucketing acelera joins repetitivos y operaciones de agregación en lecturas futuras.

Ejemplo: Estrategias de Particionado

Particionado por fecha

```
# Escribir con particionado lógico
df.write \
  .partitionBy("year", "month") \
  .mode("overwrite") \
  .parquet("data/partitioned")

# Controlar número de archivos por partición
df.repartition(4, "product_category") \
  .write \
  .partitionBy("year") \
  .parquet("data/optimized")
```

Optimización de lectura

```
# Leer particiones específicas
df = spark.read.parquet("data/partitioned") \
      .filter("year = 2023 AND month = 12")

# Bucketing para joins frecuentes
df.write \
  .bucketBy(10, "customer_id") \
  .sortBy("transaction_date") \
  .saveAsTable("transactions_bucketed")
```

Regla general: Apuntar a archivos Parquet de 100-200MB para balancear paralelismo y overhead de metadatos.

Caché y Persistencia Inteligente

1

Cuándo cachear

Cache datasets que se reutilizan múltiples veces en la misma sesión. Ideal para iteraciones de ML o análisis exploratorio.

2

Niveles de almacenamiento

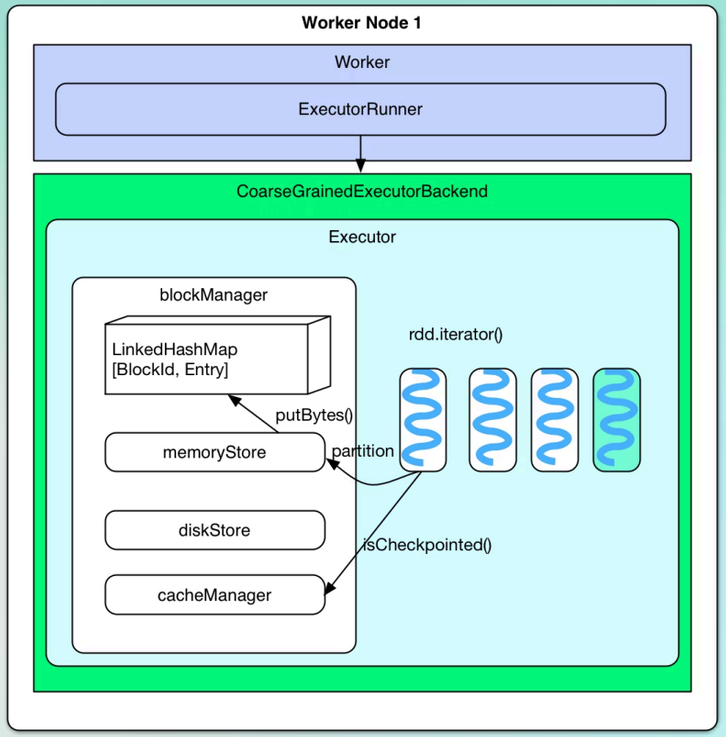
MEMORY_ONLY para datos frecuentes. MEMORY_AND_DISK para datasets grandes. DISK_ONLY para persistencia económica.

3

Gestión de memoria

Usar unpersist() para liberar memoria cuando ya no necesites el dataset. Monitor caché usage en Spark UI.

```
# Cache inteligente para reuso múltiple
expensive_df = spark.read.parquet("large_dataset.parquet") \
 .filter("status = 'active'") \
 .cache()

# Usar múltiples veces
result1 = expensive_df.groupBy("category").count()
result2 = expensive_df.filter("amount > 1000").collect()

# Liberar cuando termine
expensive_df.unpersist()
```



Gestión de Recursos y Configuración

2-4

Tasks por core

Regla inicial para balancear paralelismo y overhead de scheduling en executors

4-8GB

Memoria executor

Tamaño típico por executor, considerando overhead del 10% para operaciones internas

200MB

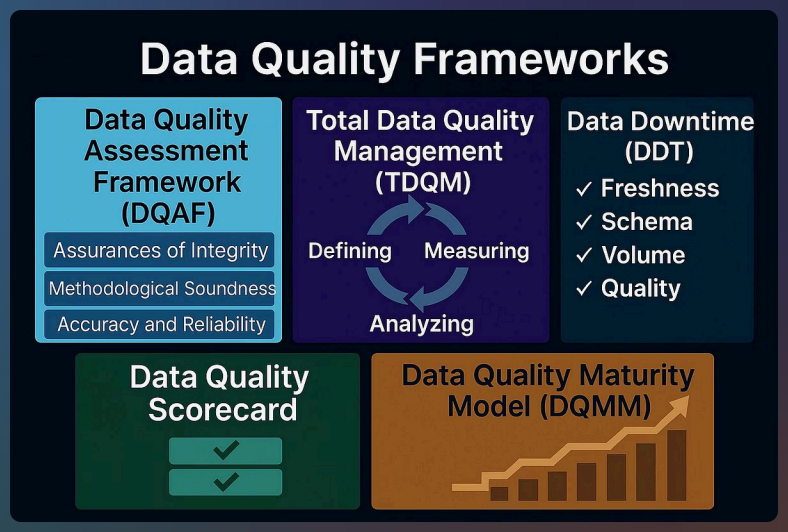
Tamaño partición

Objetivo para particiones individuales, balanceando paralelismo y eficiencia de IO

Configuración típica

```
spark.executor.instances=10
spark.executor.cores=4
spark.executor.memory=6g
spark.sql.adaptive.enabled=true
spark.sql.adaptive.coalescePartitions.enabled=true
spark.serializer=org.apache.spark.serializer.KryoSerializer
```

La configuración debe ajustarse según las características del cluster y patrones de acceso a datos. **Adaptive Query Execution (AQE)** optimiza automáticamente durante la ejecución.



Calidad de Datos en Pipelines Batch

01

Validación de esquema

Contract tests que verifican estructura y tipos de datos antes y después de transformaciones críticas

02

Profiling de columnas

Análisis estadístico automático para detectar anomalías: nulos, outliers, distribuciones inesperadas

03

Integridad referencial

Verificación de claves foráneas y constrains business entre datasets relacionados

04

Detección de skew

Identificar particiones desbalanceadas que pueden causar performance bottlenecks

```
from pyspark.sql.functions import col, isnan, when, count

# Análisis de calidad de datos
def data_quality_check(df, table_name):
 # Contar nulos por columna
 null_counts = df.select([count(when(col(c).isNull(), c)).alias(c)
 for c in df.columns])

 # Detectar skew en particiones
 partition_counts = df.rdd.mapPartitions(lambda x: [sum(1 for _ in x)]).collect()

 print(f"Quality report for {table_name}")
 print(f"Partition skew: min={min(partition_counts)}, max={max(partition_counts)}")

 return null_counts
```

Gobernanza y Seguridad

Control de accesos

Integración con sistemas de autenticación empresarial (Kerberos, LDAP) y autorización granular por tablas y columnas

Linaje de datos

Trazabilidad completa desde fuentes hasta reportes finales, facilitando auditorías y debugging de pipelines

Catálogo de metadatos

Registro centralizado de esquemas, tablas externas y contratos de datos para governanza empresarial

Manejo de PII

Enmascaramiento automático y anonimización de datos sensibles en datasets intermedios y de desarrollo

Estas prácticas se alinean con principios de **Analytics Engineering**: testing automatizado, documentación y linaje como primera clase ciudadana.

Anti-patrones Frecuentes

Usar CSV para todo

Problema: Sin esquema tipado ni estadísticas de columnas
Solución: Migrar a Parquet para datos estructurados batch

Particionado inadecuado

Problema: Demasiadas particiones pequeñas o muy pocas particiones grandes
Solución: Apuntar a 100-200MB por partición

UDFs "caja negra"

Problema: Funciones Python no optimizadas que serializan datos
Solución: Usar expresiones SQL nativas cuando sea posible

Cache indiscriminado

Problema: Cachear datasets usados una sola vez
Solución: Solo cachear con reuso confirmado

Ignorar métricas de shuffle

Problema: No monitorear Spark UI para identificar bottlenecks
Solución: Revisar regularmente Shuffle Read/Write metrics

Resumen y Próximos Pasos

Conceptos clave

Batch processing = paralelismo distribuido + evaluación perezosa + optimización automática columnar

Optimización práctica

Formato Parquet, particionado inteligente, joins optimizados y minimización de shuffle operations

Herramienta de diagnóstico

Spark Web UI como brújula principal para identificar bottlenecks y optimizar performance

Actividad de laboratorio

Implementaremos un notebook práctico con:

- Lectura optimizada de archivos Parquet grandes
- Transformaciones complejas con DataFrame API
- Joins con broadcast para tablas dimensionales
- Medición y análisis de métricas de shuffle
- Comparación de rendimiento entre diferentes estrategias

Recuerda las 3 ideas fuerza: formato eficiente (Parquet), particionado inteligente y medición constante de métricas.